

深入解析 vLLM 的 Structured Output 和 Tool Calling

蒋兴彦

DaoCloud

2025/11/15



CONTENT

目录

01 Structured Outputs 样例展示

02 Structured Outputs 工作原理

03 vLLM Tool Parser

04 vLLM Reasoning Parser

Structured Outputs 样例展示

在使用大模型时，常常会出现以下困扰

- 想让大模型整理一些文档，然后输出指定的格式，如 JSON 格式时，大模型会输出了一个 json，但在个别地方，格式上又不满足，所以又不是一个合法的 JSON。
- 想让大模型整理一些数据，输出一些 SQL 语句，但是生成的SQL 语句，复制粘贴准备执行时，又报语法错误，不是一个合法的语句。
-
-

即使写大段大段的 Prompt 来约束 大模型输出，有时依然会输出不满足格式的输出



Structured Outputs 样例展示

- 将大模型的输出，限制为符合指定的格式。
- 可通过 Python API 和兼容 OpenAI 的 API Server 调用。

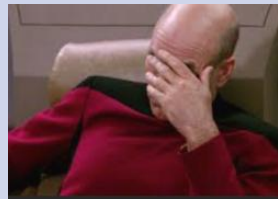


Choices

- 强制大模型从提供的选项列表中选择一个输出

例如：

Prompt: Please choose a color: red, blue, or green.

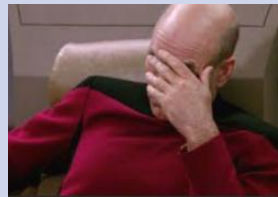
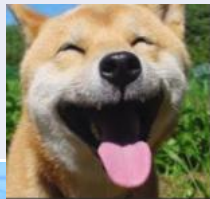
使用结构化输出?	输出结果	感受
NO	ChatGPT ▾ Please choose a color: red, blue, or green. I'll go with blue.	

Choices

- 强制大模型从提供的选项列表中选择一个输出

例如：

Prompt: Please choose a color: red, blue, or green.

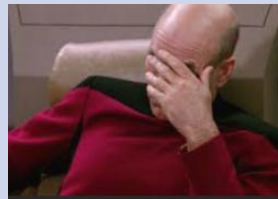
使用结构化输出?	输出结果	感受
NO	ChatGPT ▾ Please choose a color: red, blue, or green. I'll go with blue.	
choice=["red", "blue", "green"]	blue	

JSON

- 强制大模型的输出必须满足 JSON schema 格式

例如：

Prompt: Generate a JSON object with keys: name (string), age (integer), and is_student (boolean).

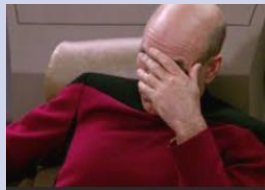
使用结构化输出?	输出结果	感受
NO	<pre>{ "name": "Alice", // a common name "age": 30, "is_student": false }</pre>	

JSON

- 强制大模型的输出必须满足 JSON schema 格式

例如：

Prompt: Generate a JSON object with keys: name (string), age (integer), and is_student (boolean).

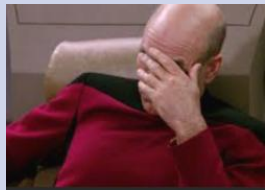
使用结构化输出?	输出结果	感受
NO	<pre>{ "name": "Alice", // a common name "age": 30, "is_student": false }</pre>	
<pre>json_schema={ "type": "object", "properties": { "name": { "type": "string" }, "age": { "type": "integer" }, "is_student": { "type": "boolean" }}, "required": ["name", "age", "is_student"], "additionalProperties": false }</pre>	<pre>{ "name": "Alice", "age": 30, "is_student": false }</pre>	

Regular Expressions

- 强制大模型的输出必须满足正则表达式约束

例如：

Prompt: Generate an email address

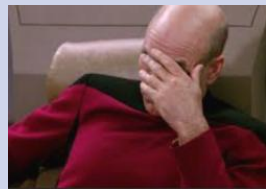
使用结构化输出?	输出结果	感受
NO	<pre>{ "email": "alice.smith@example.com" }</pre>	

Regular Expressions

- 强制大模型的输出必须满足正则表达式约束

例如：

Prompt: Generate an email address

使用结构化输出?	输出结果	感受
NO	<pre>{ "email": "alice.smith@example.com" }</pre>	
<pre>regex="[a-z0-9.]{1,20}@\\w{6,10}\\..com"</pre>	<pre>alice.smith@example.com</pre>	

Grammar

- 强制大模型的输出必须满足 语法表达式 EBNF / Lark grammars

例如:

```
root ::= select_statement
select_statement ::= "SELECT " column " from " table " where "
condition
column ::= "col_1 " | "col_2 "
table ::= "table_1 " | "table_2 "
condition ::= column "=" number
number ::= "1 " | "2 "
```

Structural Tag

- Response中位于起始与结束标记之间的JSON Schema

例如:

```
structural_tag = {  
  "type": "structural_tag",  
  "structures": [{  
    "begin": "<function=get_weather>",  
    "schema": {  
      "type": "object",  
      "properties": {  
        "city": {"type": "string"}  
      }  
    },  
    "end": "</function>"  
  }],  
  "triggers": ["<function="]
```



OpenAI-compatible API server

Optional fields in the request body:

- **choice**
- **regex**
- **son**
- **grammar**

For **structural_tag** support, set
response_format

Python API - LLM class

```
json_schema = CarDescription.model_json_schema()
```

```
structured_outputs_params_json =  
StructuredOutputsParams(json=json_schema)
```

```
sampling_params_json = SamplingParams(  
    structured_outputs=structured_outputs_params_json,  
)
```

```
llm.generate(..., sampling_params_json)
```

https://docs.vllm.ai/en/latest/getting_started/examples/examples_index.html

vLLM Integration



<> Code



```
from openai import OpenAI
client = OpenAI(
    base_url="http://localhost:8000/v1",
    api_key="-",
)
model = client.models.list().data[0].id

completion = client.chat.completions.create(
    model=model,
    messages=[
        {"role": "user", "content": "Classify this sentiment: vLLM is wonderful!"}
    ],
    extra_body={"structured_outputs": {"choice": ["positive", "negative"]}},
)
print(completion.choices[0].message.content)
```

Structured Outputs 工作原理

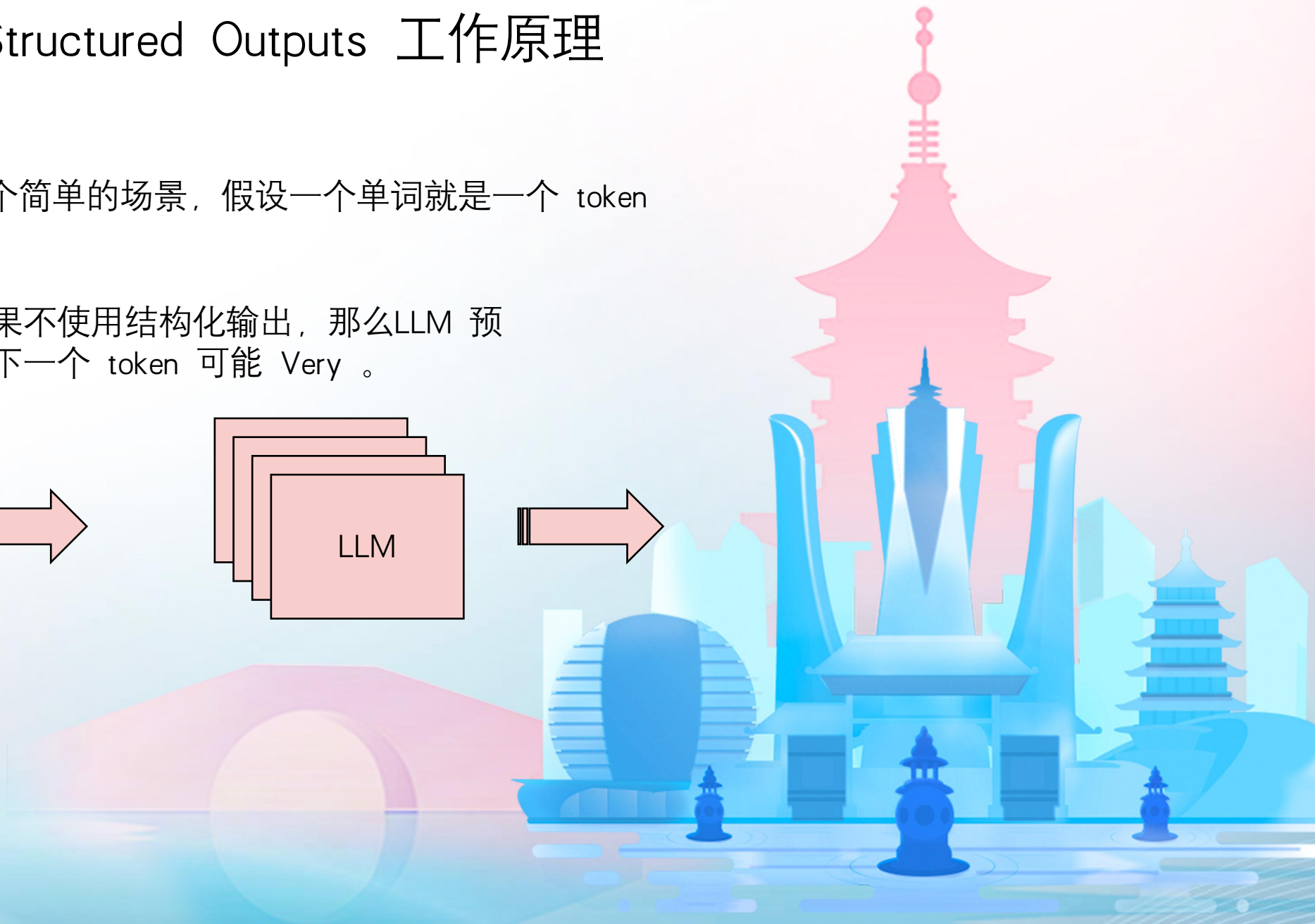
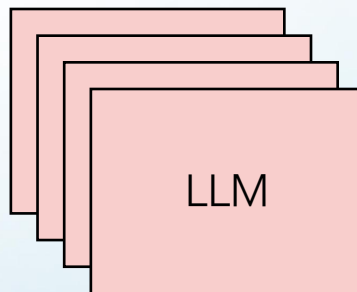


Structured Outputs 工作原理

- 考虑一个简单的场景，假设一个单词就是一个 token

如果不使用结构化输出，那么LLM 预测下一个 token 可能 Very 。

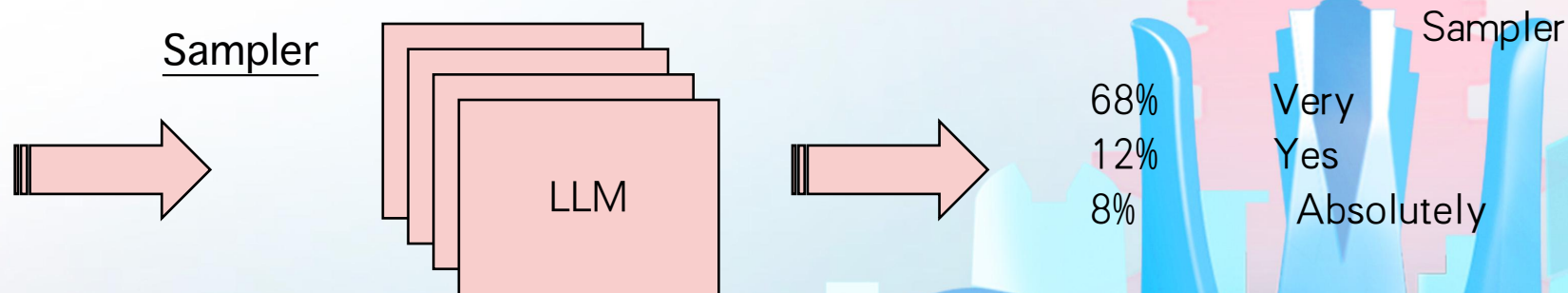
Is vLLM Fast?



Structured Outputs 工作原理

- 实际上我们从模型中得到的是一组概率，
- Sampler（采样器）会从中选择一个概率最高的一个。

Is vLLM Fast?

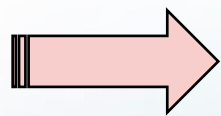
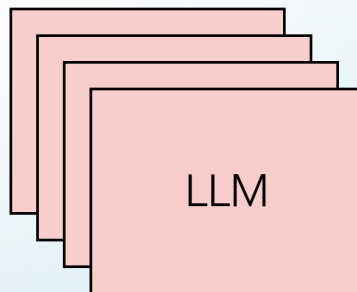


Structured Outputs 工作原理

- 使用结构化输出时，排除那些不满足规则的 token。
- choice=["Yes", "No"]

Is vLLM Fast?

Sampler

Struceured Output



68%
12%
8%

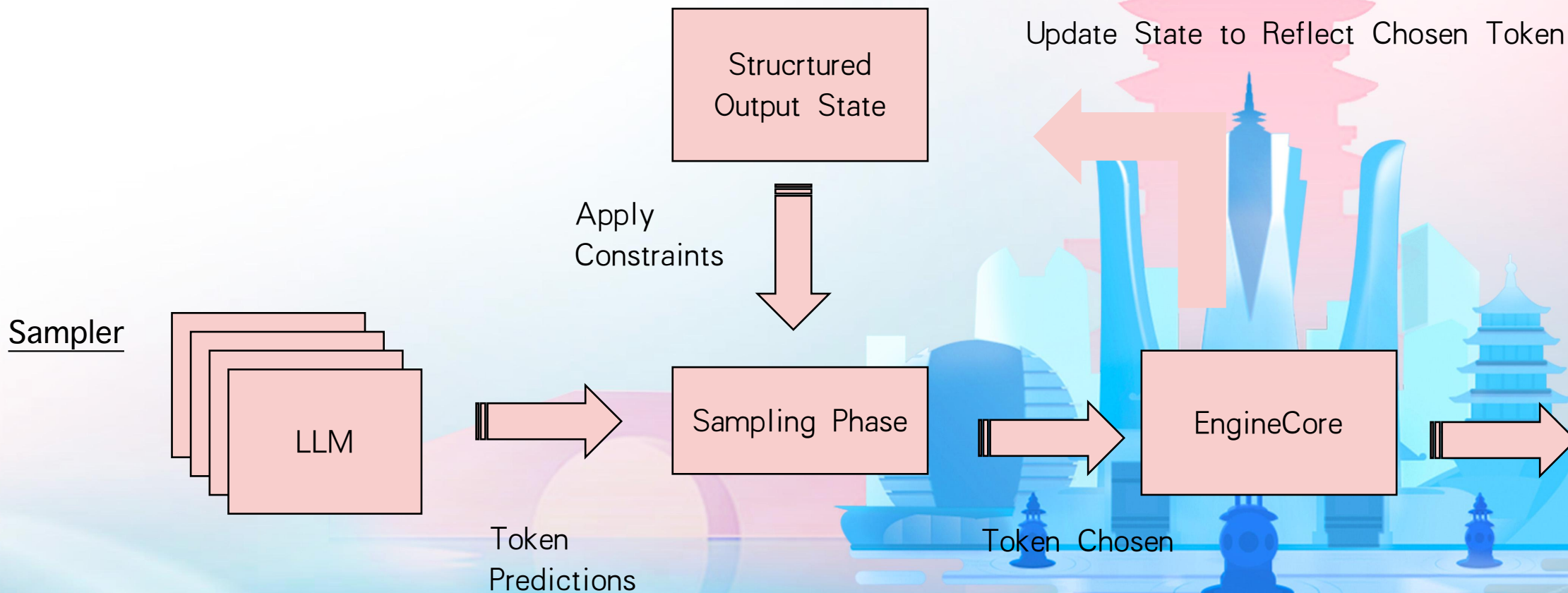
Sampler

Very
Yes
Absolutely



Structured Outputs 工作原理

- 状态机 用于跟踪在输出过程中的某个时间点哪些 token 是合法的。



Tool Parser



vLLM Tool Parser

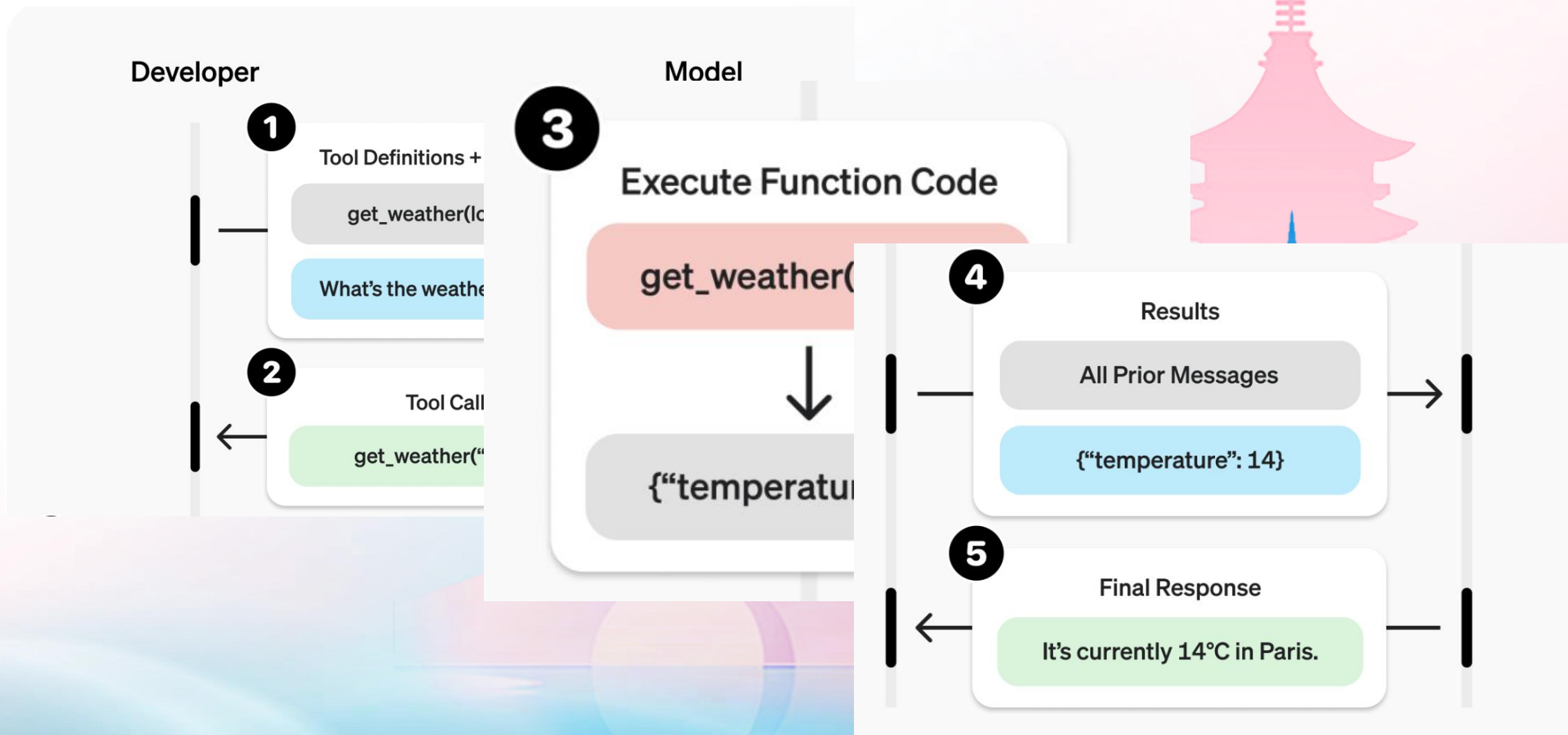
The tool calling flow

Tool calling is a multi-step conversation between your application and a model via the OpenAI API. The tool calling flow has five high level steps:

- Make a request to the model with tools it could call
- Receive a tool call from the model
- Execute code on the application side with input from the tool call
- Make a second request to the model with the tool output
- Receive a final response from the model (or more tool calls)



vLLM Tool Parser



vLLM Tool Parser

Chat template for Qwen/Qwen3-8B

[Open Playground](#)

Copy

Formatted



```
{%- if message.tool_calls %}
    {%- for tool_call in message.tool_calls %}
        {{- '<|im_end|>\n' }}
    {%- elif message.role == "tool" %}
        {%- if loop.first or (messages[loop.index0 - 1].role != "tool") %}
            {{- '<|im_start|>user' }}
        {%- endif %}
        {{- '\n<tool_response>\n' }}
        {{- content }}
        {{- '\n</tool_response>' }}
        {%- if loop.last or (messages[loop.index0 + 1].role != "tool") %}
            {{- '<|im_end|>\n' }}
        {%- endif %}
        {{- tool_call.arguments | tojson }}
        {%- endif %}
        {{- '}\n</tool_call>' }}
    {%- endfor %}
{%- endif %}
```

vLLM Tool Parser

Tool choice

By default the model will determine when and how many tools to use. You can force specific behavior with the `tool_choice` parameter.

- 1 **Auto:** (*Default*) Call zero, one, or multiple functions. `tool_choice: "auto"`
- 2 **Required:** Call one or more functions. `tool_choice: "required"`
- 3 **Forced Function:** Call exactly one specific function.
`tool_choice: {"type": "function", "name": "get_weather"}`
- 4 **Allowed tools:** Restrict the tool calls the model can make to a subset of the tools available to the model.

vLLM Tool Parser

Input

```
tool = {
  "type": "function",
  "function": {
    "name": "queryProduct",
    "parameters": {
      "type": "object",
      "properties": {
        "queryConditions": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "yield": {
                "type": "string",
                "enum": ["product", "operator"]
              },
              "value": {
                "type": "string",
                "enum": ["2"]
              }
            }
          }
        }
      }
    }
  }
}
```

tool-choice: "auto" **"required"** **"function name"**

"required"和"function name"使用结构化输出

Output :

```
"\n\n<tool_call>\n{\n  \"name\": \"queryProduct\", \"arguments\": {\n    \"queryConditions\": [{\n      \"yield\": \"productyield\", \"operator\": \">\", \"value\": \"2\"}]}\n}</tool_call>"
```

Tool Parser 使用 正则表达式 提取<tool_call></tool_call> 之间 内容

vLLM Reasoning Parser



vLLM Reasoning Parser

```
messages=[  
    {"role": "system", "content": "You are an AI"},  
    {"role": "user", "content": "1+1=?"} ],
```

Output :

```
<think>\nOkay, the user asked "1+1=?". Let me make sure I understand the question  
correctly. They want the a..... "2" as the answer,  
\n\nAlternatively, maybe they want the answer in a sentence. For example, "The  
result of 1+1 is 2." But the question is just asking for the result, so the answer is 2.  
\n\nI think the safest answer is "2" in English. So I'll provide  
that.\n</think>\n\n1+1=2
```

Reasoning Parser 使用 正则表达式 提取<think></think> 之间的内容

Thanks

